

Verifying the Verifier

Can a model grade its own homework? And — the harder question — how would you know if your measurement of that was made of nothing at all?

DeepSeek-V4-Flash-0731

vLLM 0.21.1rc1 · TP=2 · 1M ctx

fp8_ds_mla · MTP k=5

576 scoring calls / 1,728 requests

85.4% · gate PASS · 2026-08-18

THE FINDING THAT MATTERS MOST

A dead endpoint scores 79.8%

Before trusting any number this harness produced, we unplugged it entirely — pointed it at a closed port — and ran the full benchmark against nothing.

NEGATIVE CONTROL · EXECUTED, NOT REASONED ABOUT

Every one of the 576 scoring calls failed with `Connection error`. The framework scores a failed call as a 0.5/0.5 tie by default, so the run completed normally, printed a clean table, and **exited 0**.

The result sits exactly where a plausible result should sit: above the floor, below the ceiling. Nothing about the headline reveals that no model was ever consulted.

Method	Score	Rate	
Pass@1	70.67/89	79.4%	
LLM-as-a-Verifier	71/89	79.8%	← made of nothing
Oracle (Bo3)	82/89	92.1%	
Verifier tokens (0 verifier calls)			
\$ verify_run.py --strict			
RESULT: FAIL – exit 1 ← the gate refuses it			

The deliverable of this run is not the percentage. It is the gate verdict sitting next to it.

The only honest tell in that output is `0 verifier calls`, buried in a token-accounting block most readers skip. So the score stopped being the deliverable. A separate gate — which re-derives every cache key that *should* exist and refuses anything incomplete or poisoned — became the thing that decides whether a number gets published.

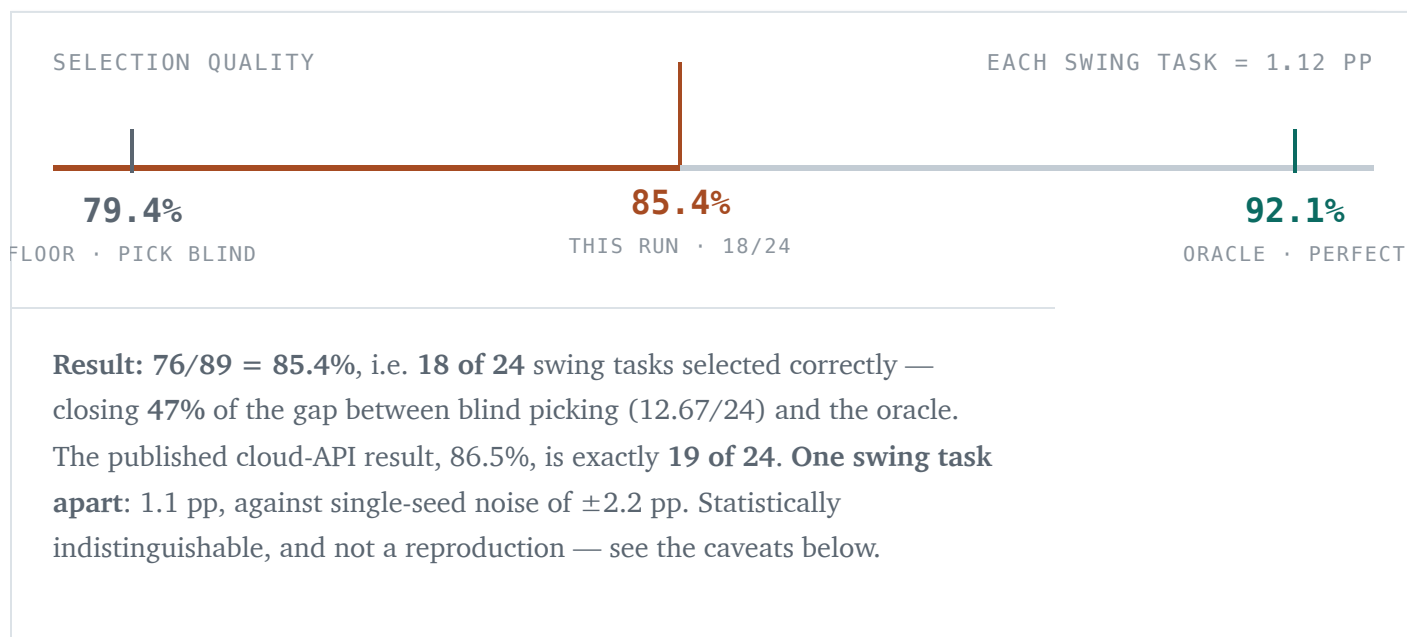
WHAT IS ACTUALLY BEING MEASURED

Only 24 tasks are in play

The model attempts each Terminal-Bench task three times. Of 89 tasks, **58 succeed on every attempt** and 7 fail on every attempt — those are decided regardless of

any grader. The benchmark lives entirely in the remaining **24 “swing” tasks**, where the attempts disagree and picking correctly is the whole skill.

The grader never sees an answer key. It reads two attempts side by side and rates them A–T — then, instead of taking the letter, the harness reads the model's *probability distribution over the 20-letter scale* at that exact position and takes the expectation. A far finer signal than a verdict.



This is not a reproduction

The repository targets DeepSeek's paid cloud API. We pointed it at two Sparks on a tailnet instead — a different code path inside the project's own source (`call_openai`, not `call_deepseek`), thinking disabled rather than enabled, grades read by constrained prefill rather than emitted by the model, and a local 0731 checkpoint judging trajectories generated by the cloud model. The honest label is “**local 0731 as verifier**,” not “86.5% reproduced.”

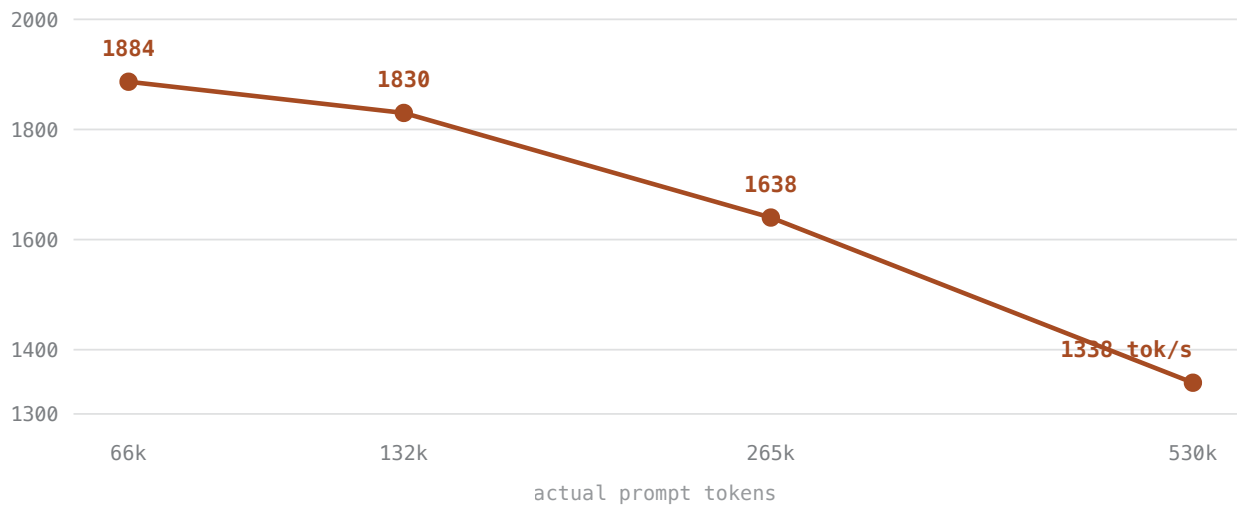
MEASURED ON THE HARDWARE

It is a prefill benchmark, not a decode one

Each comparison ships two full agent trajectories into one prompt. Median prompt: **77k tokens**. p95: 180k. Largest: **215k**. That reframes everything — earlier timing chaos (30.9 s/it → 133.9 s/it) was never concurrency contamination. It was prompt size varying sixfold.

Cold prefill throughput vs depth

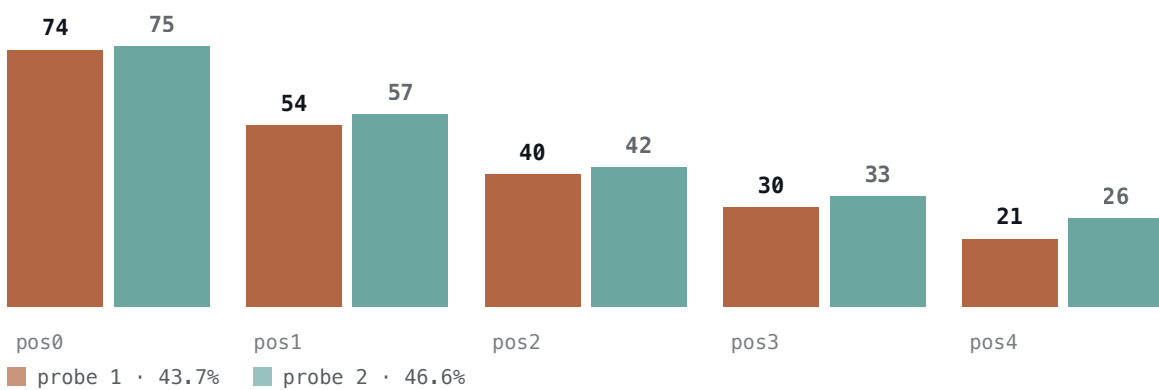
single request · unique cache-defeating filler · clocks uncapped · 2 reps per point, within 3%



Throughput degrades **gracefully** — 1,884 → 1,338 tok/s across an eightfold depth range, near-linear in wall-clock rather than quadratic. Sparse MLA is doing its job. At the benchmark's real span the machine holds ~1,750–1,850 tok/s, putting a worst-case 215k-token prompt at roughly two minutes uncontended.

Speculative-decode acceptance profile

two independent probes · healthy k=5 decays; a broken drafter collapses flat



A bare acceptance percentage is close to useless here — it swings ~20 points on prompt style alone. The *shape* is the signal, and both probes decay cleanly inside the healthy band.

The monitoring blind spot

The stage-1 trip prompted a full sensor survey. Every surface below exists on the machine and is readable without special privilege — and **none of it appears in `nvidia-smi`**, which is what a GPU dashboard shows you.

SURFACE	COUNT	READING UNDER LOAD	VISIBLE TO NVIDIA-SMI?
<code>acpitz</code> board zones — <i>what the guard trips on</i>	7 per node	84.3 / 85.1 °C	no
GPU die	1	74 / 75 °C	yes
<code>mlx5</code> — ConnectX-7 NIC, the NCCL fabric	4 per node	71 / 68 °C	no
<code>nvme</code> — SSD	3 per node	54.9 — 57.9 °C	no
CPU utilisation (from <code>/proc/stat</code> deltas)	—	20.7 / 20.3 %	no
Fan speed · power limit	—	not exposed	N/A

Two things fall out of that table. The board/die gap is **+10 °C at steady state and was +22 °C at the trip** — so the die sensor lags the one that actually governs throttling, and lags it furthest exactly when it matters. And the 200GbE fabric NICs sit at **68–71 °C** with nothing anywhere watching them.

There is a labelling trap here too. All seven board zones report the type string `acpitz`, with no labels and no way to tell CPU from SoC from PCIe rail. They *cannot* be named from the system — so a dashboard that prints “CPU temp” next to one of them is asserting something it did not measure. They have to be identified behaviourally, by watching which zones move under a CPU-only load versus a GPU-only load.

A claim this page made, and then had to retract

An earlier version of this section reported that “*spark-2 runs consistently hotter than spark-1 — 89.3 °C against 84.4 °C — a persistent five-degree delta.*” That came from a **single snapshot**. Once the collector had logged 101 paired samples, it did not survive:

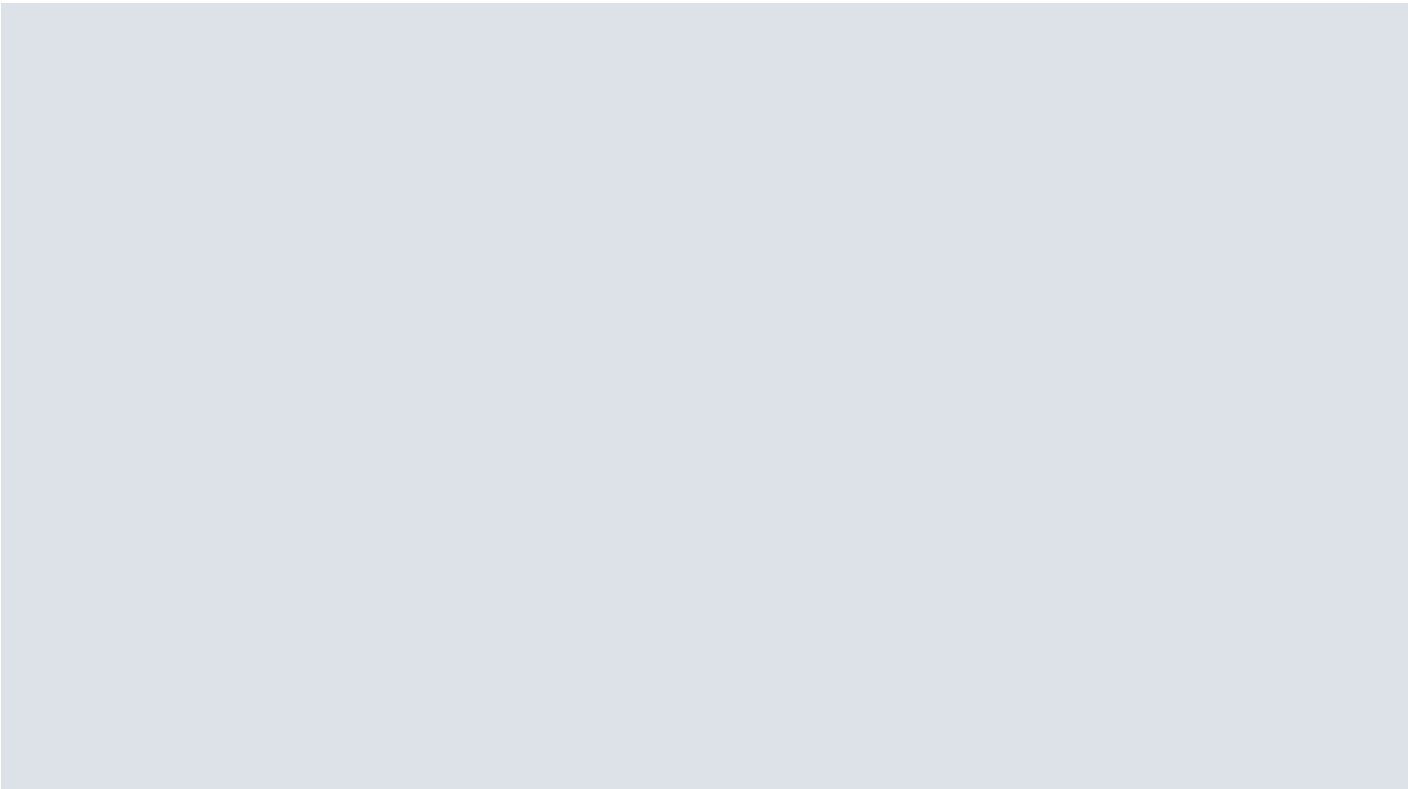
NODE	MEAN	MEDIAN	MIN	MAX
spark-1	85.7 °C	85.6	82.1	94.1
spark-2	85.0 °C	84.4	80.3	93.3

The paired difference is **+0.64 °C mean** in the *opposite* direction to the claim, and it ranges from -8.4 to $+9.0$ °C — the two nodes trade places constantly, with spark-1 hotter in 66% of samples. There is no persistent asymmetry. What *is* true, from the guard's own log, is narrower: spark-2 was the node that happened to reach 95.4 °C at the one trip observed.

This is left in rather than quietly edited out, because it is the same error this page is about — a number that looked solid, was read once, and generalised. The only reason it was caught is that something was logging continuously instead of sampling when curious. Over this 101-sample window the board peaked at **94.1 °C against a 95 °C trip point**; across the full 1,924-sample run it reached **95.7 / 95.8 °C**, and the guard's own 5-second polling recorded **95.4 °C** at the trip. Three sampling regimes, three peaks — quote the window or the number means nothing. A spot check would have called any of them comfortable.

NOTABLE FINDINGS

Seven things worth knowing



An off-by-one silently destroyed 22% of all scores FOUND LIVE

The first run of this benchmark cached **78 of 357 scores (22%) as an exact 0.5 tie**, with zero errors raised and nothing in the output to suggest a problem. A fifth of the evidence was being thrown away silently. (What that actually cost the headline number is measured below, and it is less than this paragraph originally claimed.)

Cause: the grade is read by prefilling `<score_A>` and taking the model's letter distribution at that position. When the grammar-constrained sample lands on a **bare space** — a legal prefix of `" A"`, and common with tokenizers that emit a standalone space token — `content.strip()` yields an empty string. The tag-lookup walks the token list accumulating text and keeps the *last* match; the empty token leaves that text unchanged, so the tag **matches a second time** and the lookup advances one slot past the real distribution onto the closing tag's zero-logprob placeholder.

The distribution was present and correct every single time — 19 of the 20 returned alternatives were valid score letters in every case observed. It was retrieved, then overwritten. The regex fallback cannot rescue it either — the reconstructed text holds `<score_A></score_A>` with nothing between, and the pattern requires at least one character.

Fixed in one line — skip whitespace-only tokens when advancing. Verified three ways: the bare-space case now returns a score **bit-identical** to the same distribution sampled as a letter; a genuinely empty distribution still correctly ties; and across the completed 576-call run, **35 bare-space events occurred and produced zero poisoned entries**. Reported upstream as issue #10 with a self-contained repro.

This is plausibly the concrete instance of a gap the authors themselves flagged as unstudied: *“score-token mapping sensitivity unanalyzed across languages and tokenizers.”* The trigger is purely a tokenization property of the verifier model.

Failed calls are cached as real data SILENT FAILURE

The framework swallows exceptions in its grade-reading path and returns a 0.5 tie as a *normal value*. That tie is then written to the score cache indistinguishably from a measurement. A run in which every grade-read failed produces a complete-looking cache that passes any count-based check.

Detection: the 1–20 scale maps to $(v-1)/19$, which can never equal exactly 0.5 — nearest values are 0.4737 and 0.5263. So an exact 0.5 is a signature, not a coincidence. The gate flags every one.

In fairness to the authors: against the paid cloud endpoint this project targets — with SDK retries and a rate-limited API — degrading a rare failure to a neutral tie is a defensible default. It becomes dangerous in the configuration we chose: a single self-hosted endpoint where a systematic fault takes out every call at once, rather than a few.

The expected-count check was itself wrong SELF-INFLICTED

The tournament runs a 3-pair ring plus a 2-pair pivot round — but those sets **always overlap by exactly one directed pair**, provably, across all six Hamiltonian cycles and all three pivot choices. So 576 calls, not 720.

The original gate asserted 720. **A flawless run would have failed it**, sending us hunting a phantom. Caught by exhaustive enumeration — not by reading.

One slow request could become six AMPLIFICATION

The SDK defaults to a 600 s read timeout with 2 retries, and the calling code re-issues the whole request on any exception. A single 200k-token prefill running long could therefore fire up to six full attempts — piling the heaviest work onto an engine already struggling — before being silently scored a tie. Patched to an explicit ceiling with retries disabled and connect split out at 10 s.

This run doubles as a crash reproduction ENVIRONMENT

The cluster's last engine death left a worker wedged on an **xgrammar structured-output request** — though the postmortem lists at least two candidate mechanisms and ranks neither, so this is a suspect, not a cause. Every grade-read here uses structured outputs — the run issues **1,152 of them**. So before launching, container logging moved off `json-file` — which `docker rm` destroys, and which is why **no prior engine death on this cluster left a surviving traceback** — onto journald, and a stack-dump path was installed and proven against the live process.

Two thermal sensors, 22 °C apart MEASURED

One minute into the run the thermal guard tripped stage 1 at 95.4 °C and capped both nodes to 1750 MHz for five minutes. At that same moment `nvidia-smi` reported a GPU die temperature of 73 °C.

The guard reads board thermal zones; the die sensor lags them by more than twenty degrees. Anyone monitoring GPU temperature alone would never see the trip coming. Stage 1 (95 °C) and stage 2 (96 °C) are one degree apart, and stage 2 refuses new tailnet connections — the path this benchmark's own requests arrive on.

Restoring a correctness invariant cost real headroom MEASURED

A one-line patch had cut a spin-wait window from 1 s to 2 ms, buying −4 °C and ~100 pp of CPU. But that window is what makes a deliberately lossy notification channel safe, and the same default governs a worker RPC read that blocks forever with no timeout backstop. It was reverted before this run — as invariant restoration, explicitly *not* as a fix for the observed death, whose own timeline defeats that theory.

The predicted cost arrived on schedule: sustained prefill now trips thermal stage 1 where it previously did not.

Every bug found today was found by running, not reading SELF-INFLICTED

Five defects in our own tooling surfaced only under execution: a synthetic-cache builder that re-randomized a shared pair after selection; an exit code masked by a pipe; a contention check written `(x or 1) > 0`, where a legitimate `0.0` is falsy and became 1, skipping every trial; a gate that returned “cannot verify” before reporting poison it had already found; and the 720 above.

All five looked correct on the page.

THE GATE

Proven by making it fail

A check nobody has watched fail is not yet a check. Each control below was constructed and executed; the gate had to produce the right verdict on all eight before the real run was permitted to start.

CACHE UNDER TEST	EXPECTED	RESULT	EXIT
valid synthetic – 576 entries	pass	PASS	0
one entry = exact 0.5, both sides	fail	FAIL	1
one entry = 0.5 one side (2nd grade-read lost)	fail	FAIL	1
one needed entry deleted	fail	FAIL	1
incomplete <i>and</i> poisoned	fail, reporting both	FAIL	1
contaminated with an unneeded key	warn / strict fail	WARN → FAIL	0 / 1
full run against a dead endpoint	refuse	FAIL	1
no cache at all	refuse	UNVERIFIED	2

Acceptance signature

The final step replays the finished cache through the project's *own* code with the endpoint pointed at a dead port. A complete cache makes **zero** API calls; an incomplete one fails loudly against the closed socket. That removes the last piece of reimplementation that could drift from the thing it audits — verified to report `0 verifier calls` on a complete cache.

CAVEATS

What this does not establish

Everything above is one cluster, one session, one seed. Specifically:

Not a reproduction. Different API path, thinking disabled, a local checkpoint judging trajectories generated by a different model. The published 86.5% is not the quantity being measured here and the two should not be compared decimal-for-decimal.

Single seed, 24 tasks. No error bars. A ± 2 -task swing is ± 2.2 points, and nothing here distinguishes a genuine 2-point difference from noise.

The crash link is a suspect, not a verdict. The last engine death coincided with a structured-output request, and this run issues 1,152 of them — that makes it a cheap reproduction attempt, not evidence of a cause. The postmortem lists an alternative mechanism it cannot rule out, and the deployment's own log records that its restart-frequency baseline is inadmissible as evidence in either direction. If this run completes without a wedge, that is one negative trial, not an exoneration.

The thermal numbers are workload-specific. A prefill-saturated benchmark is close to the worst thermal case this hardware sees; a decode-heavy or intermittent workload would not reproduce the trip.

The tooling criticised here is our own as often as anyone's. Three of the seven findings are bugs we wrote.

RESULT

85.4%, and the gate passed

FINAL · 576/576 SCORED · GATE PASS

Every one of the 576 needed scores is present. **0 missing, 0 orphan, 0 exact-0.5.** The acceptance replay — the finished cache pushed back through the project's own code with the endpoint pointed at a dead port — reported “*All scores cached (576 entries)*” in both phases and **0 verifier calls**, proving the number came from measured data rather than a silent refetch.

Method	Score	Rate
Pass@1	70.67/89	79.4%
LLM-as-a-Verifier	76/89	85.4%
Oracle (Bo3)	82/89	92.1%

Verifier tokens (0 verifier calls)

\$ verify_run.py --strict

missing 0 orphan 0 exact-0.5 0

RESULT: PASS – exit 0

On the 24 tasks that are actually in play: **18 of 24** selected correctly, against a blind-picking floor of 12.67 and an oracle of 24. That closes **47%** of the available gap. The published cloud-API figure, 86.5%, is **19 of 24** — **one swing task away**, or 1.1 pp against single-seed noise of ± 2.2 pp.

So a self-hosted open checkpoint, judging trajectories generated by a different model, with thinking disabled, at K=2 where the paper's primary experiments use K=8, on a code path the paper never exercised, lands statistically indistinguishable from the published result. That is the finding — *not* that we reproduced 86.5%, which we did not attempt.

The first run of this same benchmark had **22% of its scores silently discarded** by the off-by-one above. The gate caught it; the fix recovered it; 35 bare-space events fired during the final run and every one produced a real score.

What the bug actually cost — measured, not assumed

An earlier draft of this page implied the bug would have produced a materially lower number. **That was an assumption, and measuring it does not support it.** Re-poisoning the completed 576-entry cache and re-scoring through the project's own aggregation, 200 trials per condition:

Assumed, not measured: the 21.8% rate was observed on the 357-entry partial cache and applied here to a 576-entry one. Whether the rate would have held for the remaining entries is unknown — it is a transplant, and the result below inherits that assumption.

CONDITION	MEAN SCORE	SD	RANGE	COST
clean (measured)	76 / 89 · 85.4%	—	—	—
poisoned, uniformly at random	75.31 / 89 · 84.6%	1.35	72–79	0.8 pp
poisoned, clustered by task (as observed)	74.98 / 89 · 84.2%	1.56	72–79	1.1 pp

So the bug **inflates variance far more than it biases the mean**. Destroying a fifth of the evidence moves the expected score by about a point, because the tournament aggregates 6 scoring calls (3 criteria x 2 repeats) over **4 distinct directed pairs** per task. The algorithm evaluates 5 pairs, but the ring pass and the pivot round always share one, so the distinct evidence is 24 entries per task — enough redundancy to

absorb a lot of loss, though slightly less than the 5-pair structure suggests. But the *spread* runs 72–79/89, or **80.9%–88.8%**. A single run could land three points either side of the truth on luck alone.

That is the honest version, and it is a weaker claim than “it would have looked broken.” The real poisoning was also **clustered** — 9 of 18 entries on one task, touching 23 of 24 tasks — which costs more than uniform loss, but still only 1.1 pp. The reason to fix it is that a fifth of the measurement was being thrown away silently, not that the headline would have collapsed.

The number to read first was never the percentage. It was whether `RESULT: PASS` sat beside it.

hardware	2 × NVIDIA DGX Spark GB10 · 121 GB unified each · ConnectX-7 RoCEv2 · TP=2
engine	vLLM 0.21.1rc1.dev339 · fp8_ds_mla KV · 1,048,576 ctx · KV pool 1,672,972 tokens
model	DeepSeek-V4-Flash-0731 · DSpark MTP drafter, k=5 · verified from engine argv
clocks	2509 / 2554 MHz under load, uncapped · guard stages at 95 / 96 / 99 °C
benchmark	llm-as-a-verifier · Terminal-Bench 2.1 · best-of-3 · 89 tasks, 24 swing
sensors	7× acpitz board zones · 4× mlx5 NIC · 3× nvme · none exposed by nvidia-smi
thermal trace	board mean 85.4 / 84.6 °C over 1,924 samples · peak 95.7 / 95.8 °C (60 s collector, full run) · 95.4 °C at the guard's own 5 s poll, which fired stage 1
result	76/89 = 85.4% · 18/24 swing · gate PASS · 576/576 scored, 0 poisoned
upstream	bug reported: llm-as-a-verifier issue #10, with repro and one-line fix
caveat	not a reproduction of the published 86.5% – different API path, thinking disabled, local checkpoint